

Research Notes on Word2vec

Word2vec

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{j \neq 0} \log p(w_{t+j} | w_t)$$

>> Section 1

Parameters and model quality The use of different model parameters and different corpus sizes can greatly affect the quality of a word2vec model. Accuracy can be improved in a number of ways, including the choice of model architecture (CBOW or Skip-Gram), increasing the training data set, increasing the number of vector dimensions, and increasing the window size of words considered by the algorithm. Each of these improvements comes with the cost of increased computational complexity and therefore increased model generation time. In models using large corpora and a high number of dimensions, the skip-gram model yields the highest overall accuracy, and consistently produces the highest accuracy on semantic relationships, as well as yielding the highest syntactic accuracy in most cases. However, the CBOW is less computationally expensive and yields similar accuracy results. Overall, accuracy increases with the number of words used and the number of dimensions. Mikolov et al. report that doubling the amount of training data results in an increase in computational complexity equivalent to doubling the number of vector dimensions. Altszyler and coauthors (2017) studied Word2vec performance in two semantic tests for different corpus size. They found that Word2vec has a steep learning curve, outperforming another word-embedding technique, latent semantic analysis (LSA), when it is trained with medium to large corpus size (more than 10 million words). However, with a small training corpus, LSA showed better performance. Additionally they show that the best parameter setting depends on the task and the training corpus. Nevertheless, for skip-gram models trained in medium size corpora, with 50 dimensions, a window size of 15 and 10 negative samples seems to be a good parameter setting.

>> Section 2

Sub-sampling High-frequency and low-frequency words often provide little information. Words with a frequency above a certain threshold, or below a certain threshold, may be subsampled or removed to speed up training.

>> Section 3

Analysis The reasons for successful word embedding learning in the word2vec framework are poorly understood. Goldberg and Levy point out that the word2vec objective function causes words that occur in similar contexts to have similar embeddings (as measured by cosine

similarity) and note that this is in line with J. R. Firth's distributional hypothesis. However, they note that this explanation is "very hand-wavy" and argue that a more formal explanation would be preferable. Levy et al. (2015) show that much of the superior performance of word2vec or similar embeddings in downstream tasks is not a result of the models per se, but of the choice of specific hyperparameters. Transferring these hyperparameters to more 'traditional' approaches yields similar performances in downstream tasks. Arora et al. (2016) explain word2vec and related algorithms as performing inference for a simple generative model for text, which involves a random walk generation process based upon loglinear topic model. They use this to explain some properties of word embeddings, including their use to solve analogies.

>> Section 4

Approach Word2vec is a group of related models that are used to produce word embeddings. These models are shallow, two-layer neural networks that are trained to reconstruct linguistic contexts of words. Word2vec takes as its input a large corpus of text and produces a mapping of the set of words to a vector space, typically of several hundred dimensions, with each unique word in the corpus being assigned a vector in the space. Word2vec can use either of two model architectures to produce these distributed representations of words: continuous bag of words (CBOW) or continuously sliding skip-gram. In both architectures, word2vec considers both individual words and a sliding context window as it iterates over the corpus. The CBOW can be viewed as a 'fill in the blank' task, where the word embedding represents the way the word influences the relative probabilities of other words in the context window. Words which are semantically similar should influence these probabilities in similar ways, because semantically similar words should be used in similar contexts. The order of context words does not influence prediction (bag of words assumption). In the continuous skip-gram architecture, the model uses the current word to predict the surrounding window of context words. The skip-gram architecture weighs nearby context words more heavily than more distant context words. According to the authors' note, CBOW is faster while skip-gram does a better job for infrequent words. After the model is trained, the learned word embeddings are positioned in the vector space such that words that share common contexts in the corpus — that is, words that are semantically and syntactically similar — are located close to one another in the space. More dissimilar words are located farther from one another in the space.

>> Section 5

Research Notes on Word2vec

Word2vec

BioVectors An extension of word vectors for n-grams in biological sequences (e.g. DNA, RNA, and proteins) for bioinformatics applications has been proposed by Asgari and Mofrad. Named bio-vectors (BioVec) to refer to biological sequences in general with protein-vectors (ProtVec) for proteins (amino-acid sequences) and gene-vectors (GeneVec) for gene sequences, this representation can be widely used in applications of machine learning in proteomics and genomics. The results suggest that BioVectors can characterize biological sequences in terms of biochemical and biophysical interpretations of the underlying patterns. A similar variant, dna2vec, has shown that there is correlation between Needleman–Wunsch similarity score and cosine similarity of dna2vec word vectors.

>> Section 6

The idea of skip-gram is to represent each word with a vector, such that it is possible to predict the vectors of its neighbors using the vector of a word. The architecture is still linear-linear-softmax, the same as CBOW, but the input and the output are switched. Specifically, for each word w_i in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w_i . The objective of training is to maximize $\sum_i \sum_{j \in N} \ln \Pr(w_j + i | w_i)$. In full formula, the loss function is $-\sum_i \sum_{j \in N} \ln \left(\sum_{w'} e^{v_{w_j+i} \cdot v_{w_i}} \cdot \sum_{w'} e^{v_{w'} \cdot v_{w_i}} \right)$. Same as CBOW, once such a system is trained, we have two trained matrices V, V' . Either the column vectors of V or the row vectors of V' can serve as the dictionary. It is also possible to simply define $V' = V^T$, in which case there would no longer be a choice. Essentially, skip-gram and CBOW are exactly the same in architecture. They only differ in the objective function during training.